Edugenie- Project Development Phases

3. Project Design Phase

The project design phase defines the architecture and structure of the EduGenie system.

System Architecture

EduGenie follows a modular client-server architecture.

Main Components

User → HTML/CSS Frontend → FastAPI Backend → AI Modules → Gemini/Local AI → Response → User

The frontend collects the user's request and sends it to the FastAPI backend. The backend identifies the requested operation and forwards it to the appropriate module.

Major Modules

1. Q&A Module
2. Explanation Module
3. Quiz Module
4. Summary Module
5. Learning Path Module

Folder Structure

EduGenie/

├── main.py

├── explanation_module.py

├── qna.py

├── quiz_module.py

├── summary_module.py

├── learning_path.py

├── requirements.txt

├── templates/

│ └── index.html

└── static/

└── style.css

API Design

The following REST API endpoints were planned:

Endpoint

- Purpose
- /qa

Answer questions

- /explain

Explain concept

- /quiz

Generate quizzes

- /summarize

Summarize text

- /learn/recommendations

Generate learning paths

This modular design makes the system easier to maintain and extend.

- **Qna.py**

app > services > qna.py

```python
from .gemini_client import ask_gemini


def answer_question(question: str) -> str:

    if not question.strip():
        return "Please enter a question."

    prompt = f"""
    You are EduGenie AI.
    Answer the student's question clearly and simply.

    Question:
    {question}
    """

    return ask_gemini(prompt)
```

- **Learning_path.py**

app > services > learning_path.py

```python
from .gemini_client import ask_gemini


def create_learning_path(subject: str):

    if not subject.strip():
        return "Please enter a subject."

    prompt = f"""
    Create a simple learning path for:

    {subject}

    Include:
    1. Beginner topics
    2. Intermediate topics
    3. Advanced topics
    4. Revision
    5. Practice
```

- **Quiz_module.py**

app > services > quiz_module.py

```python
from .gemini_client import ask_gemini


def generate_quiz(topic: str):

    if not topic.strip():
        return "Please enter a topic."

    prompt = f"""
    Create 5 multiple-choice questions
    for a student about:

    {topic}

    Include:
    - Question
    - 4 options
    - Correct answer
    """
```

- **Summary_module.py**

```python
from .gemini_client import ask_gemini


def summarize_text(text: str) -> str:

    if not text.strip():
        return "Please enter text to summarize."

    prompt = f"""
    Summarize the following study material
    in simple student-friendly points:

    {text}
    """

    return ask_gemini(prompt)
```

- **Explanation_module.py**

```python
from .gemini_client import ask_gemini


def explain_topic(topic: str) -> str:

    if not topic.strip():
        return "Please enter a topic."

    prompt = f"""
    Explain this topic to a student in simple language:

    {topic}

    Give:
    1. Meaning
    2. Important points
    3. Simple example
    """
```